

# Inside coding agents

Gemini CLI / Codex / OpenHands SDK

Architecture / Repositories / Topic comparison

50-minute technical walkthrough

# Four layers of an agent runtime

## Interface

Renders events and collects input.  
A visible status is a view of runtime state.

## Session controller

Owens history and pending input.  
Builds requests and decides continuation.

## Tool runtime

Validates calls, applies policy, schedules work.  
Correlates each outcome with its call.

## Execution environment

Runs processes and service requests.  
Enforces the configured access limits.

Conceptual responsibilities; package boundaries differ by implementation.

# One task across several model requests

## 01 Explore

Request A reads source and tests. Results return with call identities.

## 02 Diagnose

Request B runs the failing test. A failed assertion is execution evidence.

## 03 Change

Request C proposes an edit. Admission and execution update the workspace.

## 04 Verify

Request D reruns the test. History must retain what ran and its outcome.

Illustrative parser fix: preserve the public API across every model request.

# Gemini CLI: a TypeScript workspace

## 📁 google-gemini/gemini-cli

- 📁 packages/cli/src/  
Terminal interaction and UI
- 📁 packages/core/src/  
Model loop, tools, policy and scheduling
- 📁 packages/sdk/  
Programmatic interface
- 📁 packages/vscode-ide-companion/  
Editor integration
- 📁 package.json  
npm workspace and runtime requirements

## Languages

TypeScript  
TSX in the terminal UI

## Runs as

Node.js 20+  
npm workspaces

## Build manifests

package.json  
TypeScript build configuration

Selected folders from the inspected repository. Languages describe these components, not a percentage breakdown.

# Codex: Rust core with package wrappers

## openai/codex

- codex-rs/core/  
Session loop, tools and execution policy
- codex-rs/tui/  
Terminal interface
- codex-rs/app-server/  
Server interface for clients
- codex-cli/  
npm packaging and JavaScript launcher
- sdk/typescript/  
TypeScript client SDK

## Languages

Rust: core and terminal UI  
JavaScript: npm launcher  
TypeScript: client SDK

## Runs as

Native Rust executable  
npm is one distribution route

## Build manifests

Cargo.toml  
package.json

Selected folders. The npm launcher is separate from the Rust agent runtime.

# OpenHands: four Python packages and a client

## OpenHands/software-agent-sdk

- openhands-sdk/  
Agent, conversation, events and base tools
- openhands-tools/  
Built-in terminal, editing and other tools
- openhands-workspace/  
Additional execution environments
- openhands-agent-server/  
Remote agent API server
- clients/typescript/  
Browser-compatible API client

## Languages

Python: SDK and server  
TypeScript: API client

## Runs as

Python packages  
CLI is packaged separately

## Build manifests

pyproject.toml + uv.lock  
Client: package.json

Selected SDK folders, retrieved 10 Sep 2026. The CLI is a separate application built on the SDK.

# Installing and launching the agents

## Gemini CLI

Node.js 20+ and npm

```
npm install -g @google/gemini-cli
```

Launch: `gemini`

## Codex CLI

Node.js and npm for this installation route

```
npm install -g @openai/codex
```

Launch: `codex`

## OpenHands CLI

uv and Python 3.12+; use WSL on Windows

```
uv tool install openhands --python 3.12
```

Launch: `openhands`

After installation: enter your project directory, launch the CLI, then configure model access. Commands checked 10 Sep 2026.

# Who owns the conversation?

## Gemini CLI

GeminiClient coordinates requests. GeminiChat maintains the provider-facing chat interaction.

## Codex

Session owns ongoing state. TurnContext and StepContext scope the work within that session.

## OpenHands SDK

Conversation owns lifecycle and state. Agent performs a step using the event history.

Compare ownership and lifetime. These objects are not one-to-one equivalents.

# The shared tool-call lifecycle

- |                     |                                                                     |
|---------------------|---------------------------------------------------------------------|
| <b>01 Construct</b> | Resolve the tool and validate its arguments before any side effect. |
| <b>02 Authorize</b> | Apply policy and obtain confirmation when required.                 |
| <b>03 Execute</b>   | Admit eligible work, run it and observe cancellation.               |
| <b>04 Record</b>    | Connect the result to its call and prepare the next model request.  |

Shared responsibilities across Gemini CLI, Codex and OpenHands. Concrete types differ.

# How model responses become runtime events

## Gemini CLI

Turn.run consumes the stream. It emits typed content, pending-call, retry and cancellation events.

## Codex

Sampling processes model output and dispatches tool work. Step context connects that work to the turn.

## OpenHands SDK

Agent.step / astep parses model output into message and action events before execution.

Receiving a tool request and executing it are separate lifecycle steps.

# When does the agent ask the model again?

## Gemini CLI

GeminiClient.sendMessageStream coordinates requests, bounded continuation and context checks.

## Codex

`needs_follow_up = model_needs_follow_up || has_pending_input`. Other branches still apply.

## OpenHands SDK

Conversation advances agent steps. Pending actions, observations and finish behavior affect progress.

Provider response completion and task completion are different boundaries.

# A tool definition becomes an invocation

## Gemini CLI

`build(params)` validates schema and values, then creates a `ToolInvocation` with `execute` behavior.

## Codex

Router resolves name and payload. Registry selects a handler and passes the retained context.

## OpenHands SDK

`ToolDefinition` declares `action_type`, `observation_type` and `executor`. Arguments become a typed action.

Schema discovery does not by itself establish an executable capability.

# Approval and sandboxing

## Validation

Does the call satisfy the schema and value rules?

Well-formed arguments can still request risky work.

## Sandbox enforcement

Which paths and network destinations are reachable?

Restrictions act on the execution environment.

## Authorization

May this action run under the configured policy?  
Approval should describe the concrete invocation.

## Retry policy

What changed after a denied or failed attempt?  
A retry needs a permitted execution path.

All three distinguish permission decisions from execution-environment restrictions.

# Where execution policy enters the path

## **Gemini CLI**

Invocation confirmation and scheduler policy checks happen before scheduled execution.

## **Codex**

Orchestrator resolves approval, execution restrictions and permitted retries for tools using that path.

## **OpenHands SDK**

Security analysis and confirmation can hold actions. Workspace configuration selects the environment.

Policy decisions and environmental enforcement are separate responsibilities.

# How each runtime admits concurrent tools

| System                   | Verified admission mechanism                                                                    | Consequence to inspect                                                                                        |
|--------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Gemini CLI</b>        | Batch contiguous eligible calls.<br>Execute once all active calls are<br>scheduled or terminal. | An approval wait can hold the batch.<br>Edits / update_topic stay serial.<br>wait_for_previous adds ordering. |
| <b>Codex</b>             | Tool metadata selects an RwLock<br>read guard or write guard.                                   | Calls share one admission gate.<br>Exclusive work excludes other calls<br>through that gate.                  |
| <b>OpenHands<br/>SDK</b> | Worker limit plus locks selected<br>from declared tool resources.                               | Lock conflicts constrain overlap.<br>Result collection preserves<br>the input action order.                   |

Gemini scheduler.ts: `_processNextItem` and `_isParallelizable`. Codex tools/parallel.rs.

# What makes two calls conflict?

## Gemini CLI

Edits and `update_topic` stay serial. `wait_for_previous` adds an explicit ordering boundary.

## Codex

`supports_parallel` selects shared read or exclusive write admission through one `RwLock<>`.

## OpenHands SDK

Undeclared: tool-name lock. Empty declaration: no resource lock. Keys: lock those resources.

Concurrency correctness depends on eligibility metadata and resource declarations.

# Waiting, execution and result order

## Gemini CLI

`allReady` requires active calls to be scheduled or terminal. A confirmation wait can hold a batch.

## Codex

Calls wait for the admission guard. Handler timing starts after acquisition. Results carry call identity.

## OpenHands SDK

Parallel work returns events by action ID. Batch emission follows the original action order.

Admission order, completion order and conversation order can differ.

# Cancellation must reconcile finished work

## Gemini CLI

Turn checks AbortSignal. Scheduler cancellation preserves terminal calls and cancels queued work.

## Codex

Keep terminal or finished results. Otherwise abort dispatch, then await the actual outcome.

## OpenHands SDK

Conversation manages pause and termination. Remote control also requires server-state reconciliation.

The comparison covers observed boundaries, not identical cancellation guarantees.

# How results re-enter the conversation

## Gemini CLI

ToolResult separates llmContent, returnDisplay and optional error information.

## Codex

Result payloads retain call\_id so the controller can correlate them with dispatched requests.

## OpenHands SDK

Typed observations become ObservationEvents related to ActionEvents. Batch order stays explicit.

A correct result needs both execution evidence and a relationship to its request.

# Shared problem: finite context

## The shared constraint

History grows with every request and tool result.

Each model request has a finite context budget.

Select, summarize or truncate information before sending it.

## What must survive

Task constraints such as preserving the public API.

Evidence connecting a test result to the command and revision.

Unresolved work and valid call/result relationships.

Shared engineering requirements. Reduction mechanisms differ across the three runtimes.

# How each runtime reduces context

## Gemini CLI

ChatCompressionService selects a history split and preserves valid function-call exchanges.

## Codex

The turn controller checks context usage and coordinates compaction before further requests.

## OpenHands SDK

A configured Condenser produces a smaller event view or a condensation event for later processing.

Compression quality must be checked against task constraints and valid tool history.

# Interfaces and execution locations

## Gemini CLI

TypeScript CLI connects to the core. Model APIs and remote tools can cross separate network boundaries.

## Codex

Rust core supports terminal and application interfaces. Tool restrictions depend on the execution path.

## OpenHands SDK

LocalConversation runs in process. RemoteConversation delegates to Agent Server over HTTP/WebSocket.

Source scope: public agent runtimes. Hosted model weights and services are separate systems.

# The same failure tests apply to all three

| Scenario              | Controlled test                       | Invariant to check                      |
|-----------------------|---------------------------------------|-----------------------------------------|
| <b>Malformed call</b> | Inject bad arguments.                 | No handler side effect; useful error.   |
| <b>Approval wait</b>  | Delay one decision in a batch.        | Observe admission and execution order.  |
| <b>Cancellation</b>   | Trigger before / during / after work. | Keep known completed outcomes.          |
| <b>Remote retry</b>   | Lose response after a durable write.  | Reconcile or deduplicate the operation. |
| <b>Compaction</b>     | Compress around tool exchanges.       | Retain constraints and valid history.   |

Proposed evaluation cases. These are not benchmark results or universal product guarantees.

# What to inspect in each implementation

| System               | Useful source boundary                                           | A concrete review question                                              |
|----------------------|------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>Gemini CLI</b>    | ToolInvocation and ToolResult<br>Scheduler state transitions     | Do history and display reflect the same execution outcome?              |
| <b>Codex</b>         | Step context and dispatch<br>Admission + cancellation            | Which policy applied, and what result survived the stop request?        |
| <b>OpenHands SDK</b> | Typed actions and observations<br>Resource declarations + events | Do resource keys cover conflicts, and are results correctly correlated? |

Source visibility supports code review; task quality and latency still require separate measurement.